

VulnScope Lite (VulCheck)

System / Machine Security Scanner

Phase Integration Guide — Folder Structure, Workflow, Team Responsibilities & Deliverables

GitHub Repository : <https://github.com/sal-mah/VulCheck/tree/main>

Goal of this phase: merge the four independently developed scanners into one machine-oriented security assessment tool. The target is a host/system (IP or hostname), not only a website.

Module	Owner / Responsibility	Main Purpose
Recon	Member 1	Discover host status, OS clues, ports, services and versions.
XSS Scanner	Member 2	Test applicable web applications for reflected XSS.
SQL Injection Scanner	Member 3	Test applicable web application parameters for SQL injection.
Security Configuration	Member 4	Assess host/service and web security configuration.
Integration	Team / Integrator	Orchestrate modules, aggregate findings and produce final report .

Important: Run the scanner only against systems you own or are explicitly authorized to assess. Metasploitable 2 is suitable as an isolated lab target.

1. Phase Objective

The phase converts four separate tools into one coordinated application. Recon becomes the foundation: it identifies what exists on the target, and the other modules run only when their checks are applicable.

Target model

```
Target IP / Hostname
|
v
Recon
|
+---- OS / Host information
|
+---- Open ports
|
+---- Services / versions
|
+---- Web services detected?
|
+-----+-----+
|               |
| Yes           | No
|               |
| XSS           | XSS = SKIPPED
| SQLi          | SQLi = SKIPPED
|               |
|               |
v
Security Configuration
|
v
Results Aggregator
|
v
Final Report / JSON / CSV
```

Core principle

- The target is a machine/system, such as 192.168.64.129.
- Web scanners are conditional modules, not the definition of the whole tool.
- Every module returns a standardized result structure.
- A failure in one module must not crash the complete scan.
- The final report must identify the module, severity, evidence and recommendation for every finding.

2. Whole Project Folder Structure

```
VulnScope-Lite/
|
+-- main.py
+-- requirements.txt
+-- README.md
+-- targets.txt
|
+-- core/
|   +-- __init__.py
|   +-- scanner.py
|   +-- results.py
|   +-- reporter.py
|   +-- validators.py
|
+-- modules/
|   +-- __init__.py
|   +-- recon.py
|   +-- xss_scanner.py
|   +-- sqli_scanner.py
|   +-- security_config.py
|
```

```

+-- config/
|   +-- __init__.py
|   +-- security_rules.py
|   +-- scanner_config.py
|
+-- reports/
|   +-- .gitkeep
|
+-- tests/
|   +-- __init__.py
|   +-- test_recon.py
|   +-- test_xss.py
|   +-- test_sqli.py
|   +-- test_security_config.py
|   +-- test_integration.py
|
+-- docs/
    +-- architecture.md
    +-- module_interfaces.md
    +-- test_plan.md

```

Purpose of the main folders

Folder / File	Purpose
main.py	CLI entry point. Receives target and starts the complete scan.
core/	Shared orchestration, result normalization and reporting.
modules/	The four independent security modules.
config/	Rules and configurable scanner settings.
reports/	Generated JSON, CSV or human-readable reports.
tests/	Unit and integration tests.
docs/	Architecture, interfaces and testing documentation.

3. Integration Phase Folder Structure

```
integration_phase/
|
+-- 01_prepare/
|   +-- module_inventory.md
|   +-- interface_agreement.md
|
+-- 02_standardize/
|   +-- recon_adapter.py
|   +-- xss_adapter.py
|   +-- sqli_adapter.py
|   +-- security_config_adapter.py
|
+-- 03_integrate/
|   +-- core/
|   +-- main.py
|
+-- 04_test/
|   +-- unit/
|   +-- integration/
|   +-- lab_targets/
|
+-- 05_report/
|   +-- json/
|   +-- csv/
|   +-- final_report/
|
+-- 06_documentation/
|   +-- architecture/
|   +-- user_guide/
|   +-- demo/
```

Phase sequence

Step	Work	Exit Condition
1	Collect the four current modules.	All source files are available.
2	Agree on function names and result schema.	Every module has a common interface.
3	Move modules into modules/.	Imports and paths are clean.
4	Build the orchestrator.	main.py can run modules in sequence.
5	Connect Recon to conditional scanners.	XSS/SQLi run only when applicable.
6	Aggregate findings.	One list contains all findings.
7	Test on authorized lab machines.	Expected findings are reproducible.
8	Add JSON/CSV/reporting.	Results can be consumed by a dashboard.
9	Document and demo.	Team can explain architecture and results.

4. Standard Module Interface

Before merging code, all four modules must expose predictable inputs and outputs. Do not force every module to have identical internal code; standardize only the interface.

Recommended entry point

```
def run_<module>_scan(target, recon_data=None):
    return {
        "module": "<module_name>",
        "target": target,
        "status": "success",
        "findings": [],
        "errors": []
    }
```

Finding schema

```
{
    "module": "security_config",
    "type": "Security Header",
    "name": "Content-Security-Policy",
    "severity": "Medium",
    "title": "Missing Content-Security-Policy",
    "description": "...",
    "risk": "...",
    "recommendation": "...",
    "evidence": "...",
    "url": "..."
}
```

Module status values

Status	Meaning
success	Module completed normally.
skipped	Module was not applicable to the target.
error	Module failed, but the rest of the scan can continue.

Why this matters

- The aggregator can process every module without knowing its internal implementation.
- The final report can group findings by module.
- A module can be replaced or upgraded without rewriting main.py.
- JSON/CSV output becomes straightforward.

5. Module 1 — Recon

Purpose

Recon is the foundation of the machine-oriented scanner. It determines whether the host is reachable and identifies exposed services that guide the rest of the scan.

Main workflow

```
Target
|
+--> Validate IP / hostname
|
+--> Host reachability
|
+--> Port/service discovery
|
+--> Service/version identification
```

```

|
+--> OS clues
|
+--> Identify web services
|
+--> Return recon_data

```

Expected deliverables

- recon.py with a clean run_recon_scan() entry point.
- Host status: reachable/unreachable.
- Open ports.
- Service names and versions when available.
- OS family/OS clues when available, clearly labeled as detection confidence rather than guaranteed truth.
- Web service list for XSS/SQLi conditional execution.
- Standardized JSON-compatible result.
- Unit tests for parsing and result formatting.
- A short README explaining how Recon works and its authorized lab scope.

6. Module 2 — XSS Scanner

Purpose

XSS is a web-application check. It should execute only against web services identified by Recon or against an explicitly supplied authorized web endpoint.

Main workflow

```

Recon detects HTTP/HTTPS
|
v
Discover URLs
|
v
Discover parameters/forms
|
v
Test safe reflected-XSS cases
|
v
Verify reflection / evidence
|
v
Return findings

```

Expected deliverables

- xss_scanner.py with a standard entry point.
- Input handling for URLs discovered by Recon.
- Parameter/form discovery appropriate to the project scope.
- Controlled XSS test logic for the authorized lab.
- Evidence showing where the input was reflected.
- Severity and recommendation fields.
- Tests using mocked HTTP responses where possible.
- Clear skipped status when no applicable web service exists.

7. Module 3 — SQL Injection Scanner

Purpose

SQL injection testing is also web-application-specific. It should use Recon's discovered web targets and return structured evidence instead of simply printing raw responses.

Main workflow

```
Recon detects web application
|
v
Discover URLs / parameters
|
v
Establish normal response baseline
|
v
Controlled SQLi test cases
|
v
Compare response behavior
|
v
Evidence + severity
|
v
Return findings
```

Expected deliverables

- `sql_i_scanner.py` with a standard entry point.
- Baseline response comparison.
- Controlled test logic suitable for the authorized lab.
- Evidence and confidence information.
- Severity, description and recommendation.
- False-positive reduction logic.
- Unit tests with mocked responses.
- Skipped status when no applicable web service is found.

8. Module 4 — Security Configuration Scanner

Purpose

Your module should now be broader than a website-only header checker. It should consume Recon results and inspect applicable host/service security configuration, while retaining the web configuration checks already developed.

Recommended structure

```
security_config.py
|
+-- Host Security
|
+-- Service Security
|   +-- FTP
|   +-- Telnet
|   +-- SSH
|   +-- SMB
|   +-- Database services
|
+-- Web Security
```

```

+-- Security Headers
+-- CSP Quality
+-- HSTS Quality
+-- HTTP -> HTTPS
+-- Cookie Security
+-- CORS
+-- Information Disclosure

```

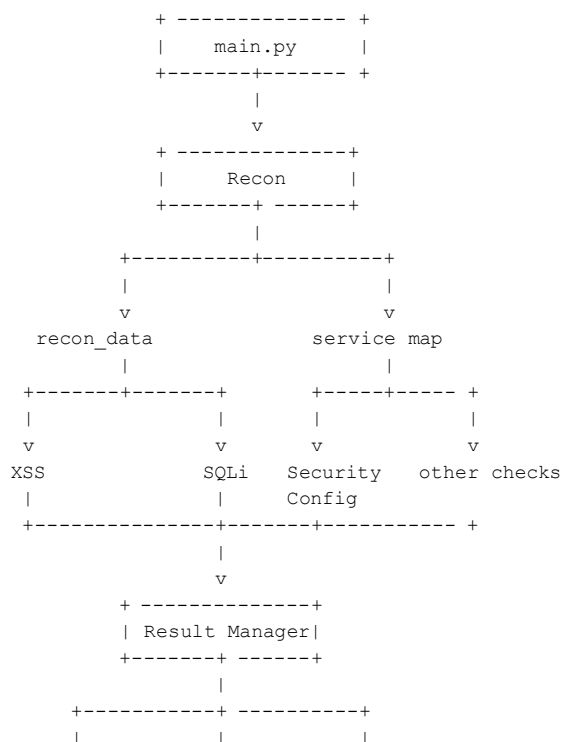
Current web checks to keep

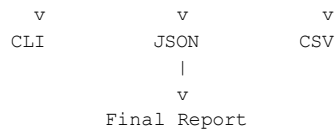
- Missing security headers.
- CSP quality: wildcard sources and unsafe-inline/unsafe-eval indicators.
- HSTS quality: max-age, includeSubDomains and preload considerations.
- HTTP-to-HTTPS redirect behavior.
- Cookie Secure, HttpOnly and SameSite.
- SameSite=None without Secure.
- Wildcard Access-Control-Allow-Origin.
- Server/X-Powered-By and related information disclosure.
- JSON/CSV output.

Next deliverables for your module

- Refactor the current security_config.py into modules/security_config.py.
- Accept recon_data so checks can be selected based on detected services.
- Separate web checks from non-web service checks.
- Return standardized findings.
- Add unit tests for headers, cookies, HSTS, CSP, redirects and service rules.
- Document exactly which checks are implemented and which are planned.

9. Integration / Orchestration Workflow





Orchestrator responsibilities

- Accept target IP/hostname.
- Validate the target format.
- Run Recon first.
- Pass Recon data to applicable modules.
- Skip modules when their prerequisites are absent.
- Catch module errors without stopping the entire scan.
- Aggregate all findings.
- Calculate summary counts and risk score.
- Export machine-readable results.

10. Complete Team Workflow

Phase	Team Action	Output
A. Individual development	Each member finishes their own scanner.	4 independent modules
B. Interface agreement	Agree on function names and result schema.	Interface document
C. Refactor	Move modules into common project structure.	Clean imports
D. Recon integration	Make Recon the source of host/service context.	recon_data
E. Conditional execution	Run XSS/SQLi only for applicable web targets.	Correct execution flow
F. Aggregation	Merge findings into one result object.	Unified findings
G. Reporting	CLI + JSON + CSV.	Reports
H. Testing	Unit + integration tests on authorized lab systems.	Test evidence
I. Documentation	Architecture, usage and demo guide.	Project documentation
J. Final demo	Run complete scan and explain findings.	Presentation/demo

11. Deliverables Matrix

Module	Code Deliverable	Testing	Documentation
Recon	recon.py + standardized result	Host/service parsing tests	Recon README + output example
XSS	xss_scanner.py	Mocked web tests	Scope + methodology + limitations
SQLi	sqli_scanner.py	Mocked response tests	Scope + evidence format
Security Config	security_config.py + security_rules.py	Header/cookie/CSP/HSTS tests	sChecks + severity rules
Core	scanner.py + results.py	Integration tests	Architecture/interface docs
Reporting	reporter.py + JSON/CSV	Export tests	Report format
Main	main.py CLI	End-to-end lab test	User guide

Final project deliverables

- Working VulnScope Lite source code.
- Organized folder structure.
- Four integrated modules.
- Standardized result schema.
- CLI interface.
- JSON and CSV exports.
- Unit and integration test suite.
- Test evidence from the authorized lab environment.
- README/user guide.
- Architecture diagram.
- Final demonstration report.

12. Team Git Workflow

```

main
|
+-- develop
    |
    +-- feature/recon
    +-- feature/xss
    +-- feature/sqli
    +-- feature/security-config
    +-- feature/integration
  
```

Rules

- Each member works primarily on their own module.
- Do not commit generated reports, caches or virtual environments.
- Pull/rebase from develop before integration when appropriate.
- Keep commits focused: one logical change per commit.
- Do not edit another member's module without coordination.
- Integration changes should happen after the interface is agreed.
- Run tests before merging.

Suggested commit examples

```
feat(recon): add service discovery result schema
feat(xss): add reflected parameter checks
feat(sqli): standardize SQLi findings
feat(security): add CSP quality checks
feat(core): add module orchestrator
feat(report): add JSON and CSV export
test(integration): add full scan test
docs: add architecture and usage guide
```

13. Testing Plan

Test Level	What to Test	Expected Result
Unit	Individual functions	Correct finding/result for known input.
Module	Each scanner independently	Module completes and returns standard schema.
Integration	Recon → XSS/SQLi/Security Config	Correct modules run based on Recon data.
Failure	Timeout, DNS failure, refused connection	Clear error; other modules continue.
Reporting	JSON/CSV	Valid machine-readable files.
Lab	Metasploitable 2 / authorized machines	Repeatable findings with evidence.
Regression	Previously fixed bugs	No reintroduction of old failures.

Metasploitable 2 demo workflow

```
Target:
192.168.64.129

1. Recon
  -> Host reachable
  -> Discover exposed services
  -> Identify HTTP if present

2. Security Configuration
  -> Analyze applicable service configuration
  -> Analyze HTTP security configuration when HTTP exists

3. XSS
  -> Run only if an applicable web application is discovered

4. SQLi
  -> Run only if an applicable web application is discovered

5. Aggregator
  -> Combine all findings

6. Report
  -> Risk summary + evidence + recommendations
```

14. Final Integration Checklist

■	All four modules are inside modules/.
■	All modules have a documented entry function.
■	All findings use the same schema.
■	Every finding identifies its module.
■	Recon runs first.
■	XSS/SQLi are conditional on web-service discovery.

■	Security Configuration consumes Recon context.
■	A module failure does not crash the complete scan.
■	CLI accepts an IP/hostname.
■	JSON export works.
■	CSV export works.
■	Risk score is calculated consistently.
■	Unit tests pass.
■	Integration tests pass.
■	Metasploitable 2 lab test is documented.
■	README explains installation and authorized use.
■	Final architecture diagram is included.

Definition of Done

The phase is complete when one command can accept an authorized target machine, run Recon, intelligently invoke applicable scanners, aggregate standardized findings, and produce a clear final report without requiring the user to manually run four separate tools.